



TP 5 – Écran d'accueil Product Explorer

Layouts Compose

1. Objectif du TP

Dans le TP précédent, vous avez créé un écran de détail produit avec les composants fondamentaux.

Dans ce TP, vous allez **continuer** la même application **Product Explorer**.

L'objectif est de construire un **écran d'accueil** en utilisant les principaux **layouts** Compose :

- **Column** ;
- **Row** ;
- **Box** ;
- **Spacer** ;
- **Surface**.

À la fin du TP, votre application affichera un écran d'accueil contenant :

- un **en-tête** ;
- une **zone de recherche** visuelle ;
- un produit **mis en avant** ;
- des **catégories** ;
- une section de **résumé** ;
- un **bouton** permettant d'aller vers le produit mis en avant, sans vraie navigation pour l'instant.

Nous restons sur des données locales.

2. Point de départ

Vous devez repartir de l'application **Product Explorer** du TP4.

Vous avez déjà normalement :

- une classe **ProductUi** ;
- une fonction **sampleProduct()** ;
- un composable **ProductDetailScreen** ;
- plusieurs petits composables comme **ProductHeader**, **ProductPriceCard**, **ProductImage**, etc.

Dans ce TP, nous n'allons pas supprimer l'écran détail.

Nous allons simplement ajouter un nouvel écran : **ProductHomeScreen** qui deviendra l'écran **affiché au lancement** de l'application.

3. Étape 1 – Afficher l'écran d'accueil depuis MainActivity

Pour commencer, nous allons remplacer l'écran **détail** par un écran **d'accueil**. Le détail produit existe toujours dans le code, mais il ne sera plus affiché directement au démarrage. Dans **MainActivity**, remplacez l'appel à **ProductDetailScreen** par :

```
Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    ProductHomeScreen(
        featuredProduct = sampleProduct(),
        onFeaturedProductClick = { // Plus tard : ouvrir le détail du produit
        },
        modifier = Modifier.padding(innerPadding)
    )
}
```



ProductHomeScreen reçoit :

- un **produit mis en avant** ;
- un **callback de clic** ;
- un **modifier**.

Il n'existe pas encore, donc Android Studio va afficher une erreur. Nous allons le créer maintenant.

4. Étape 2 – Créer la structure générale avec **Column**

Un écran d'accueil est souvent **vertical**. On a un **titre**, puis **des sections** les unes sous les autres.

Pour cela, **Column** est le composant adapté.

Ajoutez ce composable :

```
@Composable
fun ProductHomeScreen(
    featuredProduct: ProductUi,
    onFeaturedProductClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
        .fillMaxSize()
        .padding(16.dp)
    ) {
        Text(
            text = "Product Explorer",
            style = MaterialTheme.typography.headlineMedium
        )
        Text(
            text = "Découvrez les produits du moment",
            style = MaterialTheme.typography.bodyMedium
        )
    }
}
```

Column place les éléments **verticalement**.

Pour l'instant, l'écran affiche seulement un **titre** et une **phrase courte**.

Nous allons ajouter les sections progressivement.

5. Étape 3 – Ajouter un en-tête réutilisable

Le **haut de l'écran** peut devenir un composant séparé. Cela rendra **ProductHomeScreen** plus lisible.

Créez ce composable :

```
@Composable
fun HomeHeader(
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(
            text = "Product Explorer",
            style = MaterialTheme.typography.headlineMedium
        )
        Text(
            text = "Découvrez les produits du moment",
            style = MaterialTheme.typography.bodyMedium
        )
    }
}
```

Puis modifiez **ProductHomeScreen** :



```
@Composable
fun ProductHomeScreen(
    featuredProduct: ProductUi,
    onFeaturedProductClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        HomeHeader()
    }
}
```

ProductHomeScreen devient un écran qui assemble des sections.

HomeHeader gère seulement la partie haute de l'écran.

6. Étape 4 – Utiliser *Spacer* pour contrôler les espaces

Dans une interface, il ne faut pas coller tous les éléments. **Spacer** permet d'ajouter un **espace clair** entre deux zones.

Ajoutez un **espace** sous l'en-tête :

```
HomeHeader()
Spacer(modifier = Modifier.height(24.dp))
```

Spacer ne contient pas de texte ni d'image. Il sert simplement à réserver de l'espace. Cela rend l'écran plus lisible.

7. Étape 5 – Créer une zone de recherche visuelle avec *Surface*

Surface permet de créer une **zone visuelle** qui respecte le **thème Material**.

Nous allons créer une **fausse barre de recherche**. Elle ne sera pas interactive pour l'instant. Son rôle est seulement **visuel**.

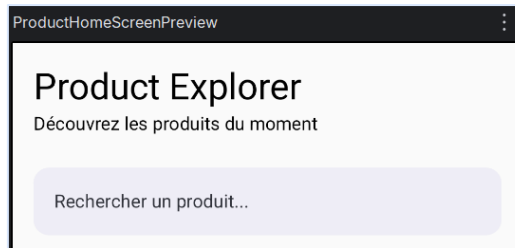
Créez ce composable :

```
@Composable
fun SearchPreviewBar(
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier.fillMaxWidth(),
        shape = MaterialTheme.shapes.medium,
        tonalElevation = 2.dp
    ) {
        Text(
            text = "Rechercher un produit...",
            modifier = Modifier.padding(16.dp),
            style = MaterialTheme.typography.bodyMedium
        )
    }
}
```

Puis ajoutez-le dans **ProductHomeScreen** :

```
HomeHeader()
Spacer(modifier = Modifier.height(24.dp))
SearchPreviewBar()
Spacer(modifier = Modifier.height(24.dp))
```

La **barre de recherche** est volontairement simple. On ne gère pas encore de **saisie** ni de **filtre**. On travaille ici le placement et la structure visuelle.



8. Étape 6 – Créer une carte de produit mis en avant

L'écran d'accueil doit afficher un produit principal.

Nous allons créer un composant `FeaturedProductSection`.

Il utilisera :

- `Surface` pour le bloc visuel ;
- `Column` pour organiser le contenu verticalement ;
- `Button` pour l'action.

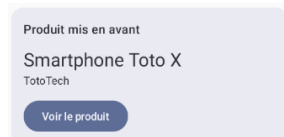
Créez ce composable :

```
@Composable
fun FeaturedProductSection(
    product: ProductUi,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier.fillMaxWidth(),
        shape = MaterialTheme.shapes.large,
        tonalElevation = 4.dp
    ) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            Text(
                text = "Produit mis en avant",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(modifier = Modifier.height(12.dp))
            Text(
                text = product.title,
                style = MaterialTheme.typography.headlineSmall
            )
            Text(
                text = product.brand,
                style = MaterialTheme.typography.bodyMedium
            )
            Spacer(modifier = Modifier.height(12.dp))
            Button(onClick = onClick) {
                Text(text = "Voir le produit")
            }
        }
    }
}
```

Puis ajoutez-le dans `ProductHomeScreen` :

```
FeaturedProductSection(
    product = featuredProduct,
    onClick = onFeaturedProductClick
)
Spacer(modifier = Modifier.height(24.dp))
```

- Le composant reçoit le produit en paramètre.
- Il ne décide pas ce qu'il se passe après le clic.
- Il signale simplement le clic avec `onClick`.





9. Étape 7 – Ajouter une ligne d'informations avec Row

Row place les éléments **horizontalement**. Nous allons l'utiliser pour afficher rapidement :

- le **prix** ;
- la **note** ;
- le **stock**.

Ces trois informations seront **sur la même ligne**.

Créez ce composable :

```
@Composable
fun ProductQuickInfoRow(
    price: Double,
    rating: Double,
    stock: Int,
    modifier: Modifier = Modifier
) {
    Row(modifier = modifier.fillMaxWidth()) {
        Text(
            text = "$price €",
            modifier = Modifier.weight(1f)
        )
        Text(
            text = "★ $rating",
            modifier = Modifier.weight(1f)
        )
        Text(
            text = "$stock en stock",
            modifier = Modifier.weight(1f)
        )
    }
}
```

Ajoutez ensuite cette ligne dans **FeaturedProductSection**, juste avant le bouton :

```
ProductQuickInfoRow(
    price = product.price,
    rating = product.rating,
    stock = product.stock
)
Spacer(modifier = Modifier.height(12.dp))
```

Row place les informations côte à côte.

weight(1f) donne une part **équivalente de largeur** à chaque élément.

Cela évite que tout soit collé à gauche.

Produit mis en avant

Smartphone Toto X

TotoTech

699.99 €

★ 4.6

34 en stock

Voir le produit

10. Étape 8 – Créer des catégories avec Row et Surface

L'accueil peut afficher **quelques catégories**.

Nous allons créer de **petites zones visuelles** avec **Surface** .

Puis nous les placerons dans une **Row**.

Créez d'abord un petit composant de **catégorie** :



```
@Composable
fun CategoryChip(
    label: String,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier,
        shape = MaterialTheme.shapes.medium,
        tonalElevation = 2.dp
    ) {
        Text(
            text = label,
            modifier = Modifier.padding(horizontal = 12.dp, vertical = 8.dp),
            style = MaterialTheme.typography.bodyMedium
        )
    }
}
```

Puis créez la section :

```
@Composable
fun CategoriesSection(
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(
            text = "Catégories",
            style = MaterialTheme.typography.titleMedium
        )
        Spacer(modifier = Modifier.height(8.dp))
        Row {
            CategoryChip(label = "Smartphones")
            Spacer(modifier = Modifier.weight(1f))
            CategoryChip(label = "Audio")
            Spacer(modifier = Modifier.weight(1f))
            CategoryChip(label = "Maison")
        }
    }
}
```

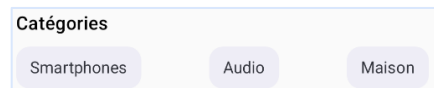
Ajoutez ensuite dans **ProductHomeScreen** :

```
CategoriesSection()
Spacer(modifier = Modifier.height(24.dp))
```

Ici, **Column** organise la section **verticalement**.

Row place les catégories **horizontalement**.

Surface donne à chaque catégorie un aspect de petit **bloc** **visuel**.



11. Étape 9 – Utiliser Box pour créer un bloc avec badge

Box permet de placer des éléments **dans la même zone**.

On peut par exemple afficher un texte principal et un petit badge dans un coin.

Nous allons créer une zone « **Offre du jour** ».

Créez ce composable :

```
@Composable
fun DailyOfferBox(
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier.fillMaxWidth()
    ) {
        Surface(
```



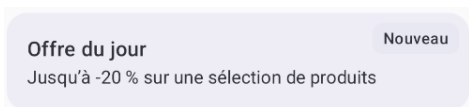
```
        modifier = Modifier.fillMaxWidth(),
        shape = MaterialTheme.shapes.large,
        tonalElevation = 2.dp
    ) {
        Column(
            modifier = Modifier.padding(16.dp)
        ) {
            Text(
                text = "Offre du jour",
                style = MaterialTheme.typography.titleMedium
            )
            Text(
                text = "Jusqu'à -20 % sur une sélection de produits",
                style = MaterialTheme.typography.bodyMedium
            )
        }
    }
    Surface(
        modifier = Modifier
            .align(Alignment.TopEnd)
            .padding(8.dp),
        shape = MaterialTheme.shapes.small,
        tonalElevation = 6.dp
    ) {
        Text(
            text = "Nouveau",
            modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
            style = MaterialTheme.typography.labelMedium
        )
    }
}
```

Ajoutez ensuite dans `ProductHomeScreen` :

```
DailyOfferBox()
```

Le premier `Surface` représente le **bloc principal**.

Le deuxième `Surface` représente le **badge**.



`Box` permet de placer le **badge en haut à droite** avec : `Modifier.align(Alignment.TopEnd)`

12. Étape 10 – Écran d'accueil final

Votre `ProductHomeScreen` doit maintenant assembler les sections. Il reste court et lisible.

Voici une version finale possible :

```
@Composable
fun ProductHomeScreen(
    featuredProduct: ProductUi,
    onFeaturedProductClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        HomeHeader()
        Spacer(modifier = Modifier.height(24.dp))
        SearchPreviewBar()
        Spacer(modifier = Modifier.height(24.dp))
        FeaturedProductSection(
            product = featuredProduct,
            onClick = onFeaturedProductClick
        )
        Spacer(modifier = Modifier.height(24.dp))
        CategoriesSection()
        Spacer(modifier = Modifier.height(24.dp))
        DailyOfferBox()
    }
}
```



L'écran principal ne contient presque pas de détails techniques.

Il assemble simplement des sections.

C'est exactement ce que l'on cherche : un écran lisible, structuré et facile à faire évoluer.

13. Preview de l'écran d'accueil

Ajoutons une **preview** pour vérifier rapidement l'écran d'accueil.

```
@Preview(showBackground = true)
@Composable
fun ProductHomeScreenPreview() {
    ProductExplorerTheme {
        ProductHomeScreen(
            featuredProduct = sampleProduct(),
            onFeaturedProductClick = {},
        )
    }
}
```

Cette **preview** vérifie l'écran d'accueil sans lancer toute l'application.

Le clic est vide pour l'instant.

14. Version complète avec des nouveaux composables

Voici une version complète possible de **MainActivity.kt** :

<https://gist.github.com/ariatsi/e714eb35cfa788b57853743457c79979> .

Adaptez le nom du package et le nom du thème à votre projet.

Et voici le répertoire GitHub avec cette version :

<https://github.com/ariatsi/ProductExplorer/releases/tag/tp5-final> .

15. Ce qu'il faut retenir

Dans ce TP, vous avez construit l'écran d'accueil de **Product Explorer**.

Vous avez utilisé :

- **Column** pour organiser l'écran **verticalement** ;
- **Row** pour placer des informations **horizontalement** ;
- **Box** pour **superposer un badge** sur un bloc ;
- **Spacer** pour contrôler les **espaces** ;
- **Surface** pour créer des **zones visuelles** cohérentes.

Vous avez aussi continué à écrire des composants réutilisables avec des paramètres, un modifier et des **callbacks**.

Dans le prochain TP, vous réutiliserez cette base pour afficher un catalogue local de produits avec des **listes** modernes.

